CHAPTER 1

DESIGNING AN AIRLINE MANAGEMENT SYSTEM

1 Getting Ready: The Airline Management System 2
2 Requirements for the Airline Management System 4
3 Use Case Diagram for the Airline Management System 5
4 Class Diagram for the Airline Management System 8
5 Sequence Diagram for the Airline Management System 19
6 Activity Diagram for the Airline Management System 21
7 Code for the Airline Management System 22
8 Summary: Designing an Airline Management System 31
9 Key Insights and Tips: Designing an Airline Management System 34

This chapter shows the design of an airline management system, detailing the requirements, UML diagrams, and implementation code for the system.

1 Getting Ready: The Airline Management System

1.1 Problem definition

An **airline management system** is a software used to efficiently manage all activities within an airline's system. Nowadays, every airline has a management system to digitize the process of scheduling flights, managing staff, ticket reservations, and performing other necessary airline management tasks. Moreover, the system tracks the number of aircraft, pilots, and their availability at airports. The system provides customers the complete flight information to view the available flights and schedule them online. The front desk officer should be able to reserve tickets, create itineraries, and process flight payments for customers, providing a smoother booking experience for those who prefer or require in-person assistance. Similarly, the admin can manage all the airline activities with the help of this system. Therefore, the airline management system facilitates its customers and administrators, controlling all operations of the airline company.

Figure 1.1: Airline management system

1.2 Expectations from the interviewee

The airline management system comprises several components, each with its own specific constraints and requirements. The following provides an overview of some of the main things that the interviewer will want to hear you discuss in more detail during the interview.

Flight reservation

Flight reservation is a crucial component of an airline's management system. The system must ensure that no two people are assigned to the same seat. The interviewer expects you to ask questions to identify how the system will complete flight reservations:

- How will the system ensure that multiple users do not have the same seat on the aircraft?
- Can one itinerary reserve multiple flights?
- Can the customer reserve the whole aircraft?

Payment handling

One of the airline management system's most significant attributes is its payment structure for its customers. This can vary, so the interviewer would expect you to ask the questions listed below:

- What payment methods are accepted, such as credit cards or cash?
- How does the customer make the payment? Does the customer pay online or in person?
- Will the customer be able to pay in advance for a flight booking, or is a just-in-time (JIT) payment method available?

Price variance

Now that we've discussed the payment methods of the airline management system, let's ask the interviewer about the pricing model. You may ask the questions listed below:

- Is the price set manually, or does the system calculate the price for each flight?
- Do all seats have the same price, or is it calculated based on the seat type?
- Do weekdays and weekends affect the price of the flight?
- Is the price of the flight affected by an increase in demand?
- How does the duration of the flight affect the payment?

Flight cancellation

In the airline management system, customers may want to cancel a flight after booking it. Therefore, you can ask the following questions:

- Can the customer cancel a flight?
- What is the time limit to cancel the flight?
- Which type of users are allowed to request a flight cancellation?

Flight scheduling

Flight scheduling is crucial for ensuring timely and conflict-free operations. The interviewer might expect you to ask:

- How is a flight schedule created and managed?
- Can a flight be delayed or rescheduled, and how is this handled in the system?
- Is there any buffer time considered between flights for aircraft and crew?

Staff and crew management

Managing pilots and crew availability is vital for operations. You may want to ask:

- How is crew availability tracked in the system?
- Can crew members be assigned to multiple flights in a day?
- What happens if a crew member becomes unavailable suddenly?

1.3 Design approach

We'll design this airline management system using the bottom-up approach. For this purpose, we will follow the steps below:

- Identify and design the smallest components first, including the seat, flight, etc.
- Use these small components to design larger components, such as an airport or an aircraft that can be composed of multiple seats.
- Repeat the steps above until the entire airline management system is designed.

1.4 Design pattern

During an interview, it is always a good practice to discuss the design patterns that an airline management system falls under. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:

Which design pattern(s) should be used to design an airline management system? Please elaborate on your choice(s).

Let's explore the requirements of the airline management system in the next lesson.

1.5

2 Requirements for the Airline Management System

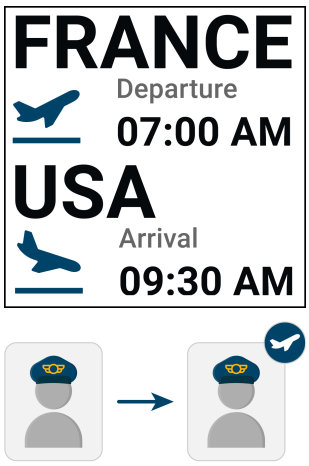
In this lesson, we'll outline the requirements for the airline management system. This is a crucial step because requirements define the scope of a problem. Therefore, accurately obtaining and thoroughly understanding them from the interviewer will make the design of the rest of the system smooth and easy.
We'll use the notational convention to identify each requirement with a unique label, "Rn," where "R" is short for requirement and "n" is a natural number.

2.1 Requirements collection

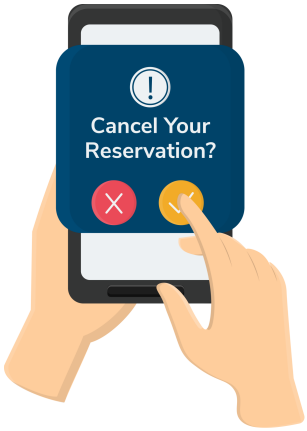
The requirements for the airline management system are defined below:

R1: Customers should be able to search for flights by date, departure, and destination airport.
R2: Customers should be able to reserve tickets for available flights and book multiple flights at once.
R3: The customer should be allowed to book multiple seats for a single flight.
R4: The system should allow customers to check flight details, such as available seats, flight schedules, and departure/arrival times.

R5: The admin should be able to add new flights and update or cancel scheduled flights.
R6: An airline should be able to own multiple aircraft, and the admin should be able to add these aircraft to the system.
R7: An airline should be able to operate its flights from different airports.
R8: The admin should be able to effectively assign pilots and crew members to flights.



R9: The customer should be able to make payments against their flight reservations.
R10: The customer should be able to cancel their previous reservations.
R11: The front desk officer should be able to reserve tickets, create itineraries, and make flight payments for the customer.
R12: The flight crew should be able to view the schedule for their assigned flights.
R13: The system should notify the customer whenever a reservation is made or canceled, or when there is an update regarding their flight.



We've identified our requirements for the problem. In the next lesson, we will define different use cases of the airline management system.

3 Use Case Diagram for the Airline Management System

Let's build the airline management system's use case diagram and understand its components. First , we'll define the different elements of our airline, followed by the complete use case diagram of the system.

3.1 System

Our system is an airline.

3.2 Actors

Now, we'll define the main actors of the airline management system.

Primary actors

- **Customer:** The customer is the primary actor for the airline. The customer can search for flights, create an itinerary, make a payment, and update or cancel the flight reservation.

- **Front desk officer:** This actor can perform all the actions that the customer can. The front desk officer can create an itinerary, make payments, update or cancel flight reservations on behalf of customers, assign seats, and search for flights.

- **Admin:** The admin is responsible for performing various operations, including adding aircraft to the system, creating or modifying flights, flight instances, and their schedules, canceling flights, and assigning crew to flights.

Secondary actors

- **System:** This actor sends notifications for flight status updates, itinerary changes, and reservation cancellations.

- **Crew:** This actor can view the schedules of the assigned flights.

3.3 Use cases

In this section, we'll define the use cases for an airline management system. We have listed the use cases according to their interactions with a particular actor.

> **Note:** You may see some use cases repeated multiple times because they are shared among different actors in the system.

Customer/front desk officer

- **Update/cancel reservation:** To update or cancel a flight reservation of the customer.
- **Login/logout:** To log in or log out of the airline system.
- **Reset password:** To reset the password of the account.
- **Create itinerary:** To create an itinerary for the customer.
- **Assign seat:** To assign a seat to the passenger for the flight.
- **Search flights:** To search for flights in the airline management system.
- **Make payment:** To pay for the itinerary or flight reservation.
- **View itinerary:** To view the details of an itinerary.
- **Cancel itinerary:** To cancel the itinerary for the customer.

Admin

- **Add aircraft:** To add a new aircraft to the airline management system.
- **Add/modify flight:** To add a new flight or modify an existing one.
- **Assign crew:** To assign crew to the flight instance.
- **Cancel flight:** To cancel the instance of a flight.
- **Search flights:** To search for flights in the airline management system.

System

- **New itinerary notification:** To send a notification of a new itinerary to the customer.
- **Flight status notification:** To send the flight status update notification to the customer.
- **Reservation cancel notification:** To send a reservation cancellation notification to the customer.

Crew

- **View flight schedule:** To view the schedule of the assigned flights.

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Associations

The table below illustrates the association between actors and their corresponding use cases.

Customer	Front Desk Officer	Admin	Crew
Cancel reservation	Cancel reservation	Add aircraft	View flight schedule
Login/logout	Login/logout	Add/modify flight	
Reset password	Reset password	Search flights	
Create itinerary	Create itinerary	Assign crew	
Assign seat	Assign seat	Cancel flight	
Search flights	Search flights		
Make payment	Make payment		
View itinerary	View itinerary		
Cancel itinerary	Cancel itinerary		

Include

Users may add passengers, create a flight reservation, or trigger a new itinerary notification when they create an itinerary.

- The "Create itinerary"use case includes the "Add passenger," " Create flight reservation," and "New itinerary notification" use cases.

When a user creates a flight reservation, they may assign a seat.

- The "Create flight reservation" use case includes the "Assign seat" use case.

When a user assigns a seat, they may proceed to make a payment.

- The "Assign seat" use case includes the "Make payment" use case.

When a user cancels a reservation, the system may notify the user and process a refund.

- The "Cancel reservation" use case includes the "Reservation cancel notification" and "Refund payment"use cases.

When a user cancels an itinerary, the system may notify the user and process a refund.

- The "Cancel itinerary" use case includes the "Reservation cancel notification" and "Refund payment" use cases.

The system may send a flight status notification when an admin adds or modifies a flight.

- The "Add/Modify flight" use case includes the "Send flight status notification" use case.

When an admin cancels a flight, the system may cancel related reservations, process refunds, and send cancellation notifications.
The "Cancel flight"use case includes the "Cancel reservation," "Refund payment," and "Send cancel notification" use cases.

Extend

- When modifying a flight instance, the admin can assign the crew to that instance. Therefore, the "Modify flight instance" use case has an extend relationship with the "Assign crew" use case.

3.5 Use case diagram

Here's the use case diagram of the airline management system:

In the next lesson, we'll discuss the class diagram with a detailed explanation of all classes and their relationships with each other.

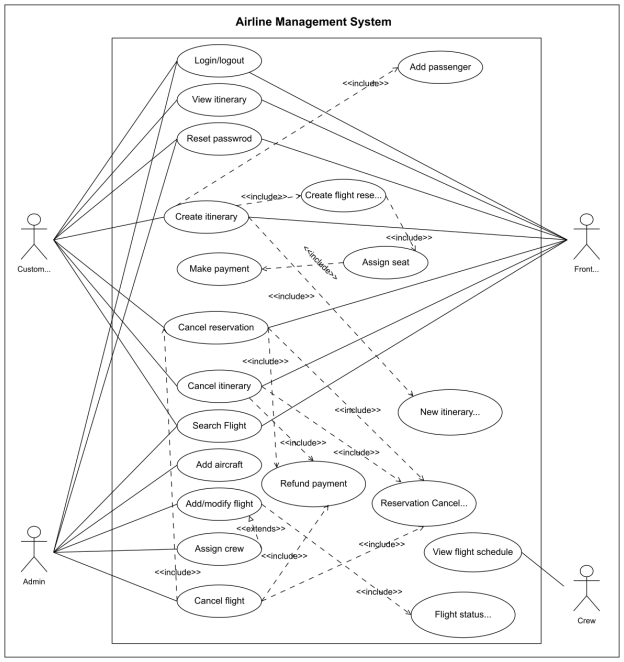


Figure 1.2: The use case diagram of the airline management system

4 Class Diagram for the Airline Management System

In the class diagram, we will first design and create the system's classes, abstract classes, and interfaces. Then , we'll identify the relationship between classes by all the requirements of the airline management system.

4.1 Components of an airline management system

In this section, we'll define the classes for an airline management system. As mentioned earlier, we will design the system using a bottom-up approach. First , we will create the classes of small components. Next , we will integrate these components and create the class diagram for the entire system.

Account

The `Account` class identifies the username and ID of an airline management system user. The class definition is represented below:

Person

A `Person` is an abstract class used to store information related to a person, such as a name, email, phone number, etc. In this class, the `Address` object type specifies the person's address. There can be four types of accounts in the system:

- `Admin`**:** This class manages the overall system, including flight and aircraft records.
- `Crew`**:** This class views assigned flights in the system.
- `FrontDeskOfficer`**:** This class manages passenger reservations.
- `Customer`**:** This class displays flight schedules and allows users to reserve or cancel flight reservations.

The relationship diagram for these classes is shown below:

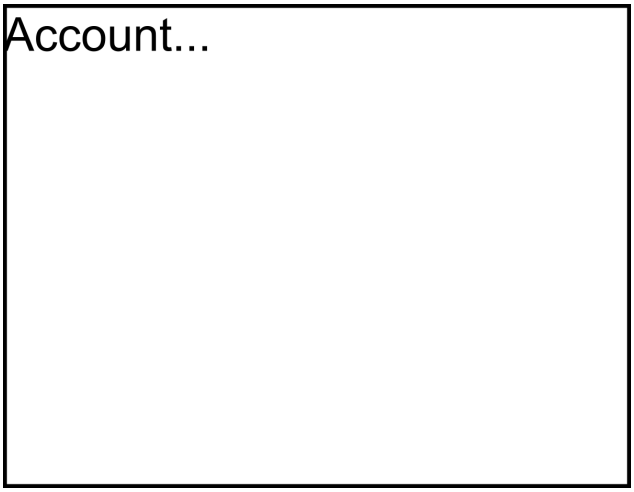


Figure 1.3: The Account class

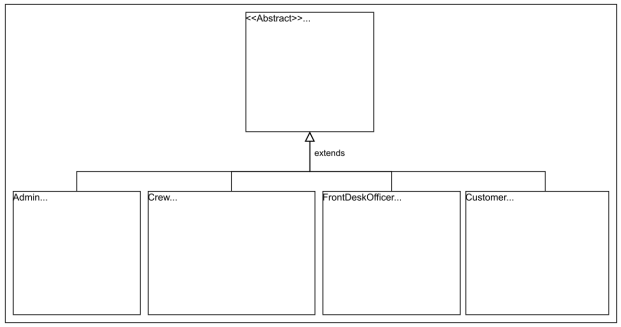


Figure 1.4: Person and its derived classes

R4: The system should allow customers to check flight details, such as available seats, flight schedules, and departure/arrival times.
R5: The system admin should be able to add new flights and aircraft. The admin should be able to cancel previous flights.
R11: The front desk officer should be able to reserve tickets, create itineraries, and make flight payments for the customer.
R12: The flight crew should be able to view the schedule for their assigned flights.

Airline, airport, and aircraft

The `Airline` class has attributes such as a name and an airline code to distinguish it from other airlines. It is a crucial component of the airline management system.
Each airline operates out of different airports. Therefore, we need the `Airport` class to keep track of all the airports in the system.
Airlines own or hire aircraft to carry out their flights. The `Aircraft` class has attributes such as name, model, and manufacturing year, among others.
Here's a visual representation of these classes:

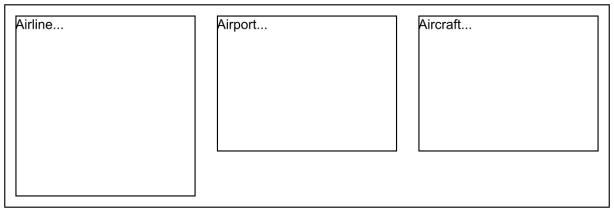


Figure 1.5: The Airline, Airport, and Aircraft classes

R6: An airline should be able to own multiple aircraft, and the admin should be able to add these aircraft to the system.
R7: An airline should be able to operate its flights from different airports.

Seat and flight seat

The `Seat` class represents a physical seat in the aircraft. It contains basic information, such as seat number, seat type, and class. The `FlightSeat` class is derived from the `Seat` class and represents the seat assigned to a specific flight instance. It contains the fare for that particular seat and its reference number.
The relationship diagram for these classes is shown below:

Flight and flight instance

The `Flight` class contains information about a particular flight, including the flight number, departure and arrival airports, and the duration of the flight. The `FlightInstance` class represents a single occurrence of a flight, since a flight can fly multiple days in a week.
These classes are represented below:
R1: A customer should be able to search for flights by date, departure, and destination airport.
R4: The system should allow customers to check flight details, such as available seats, flight schedule, and departure/arrival times.

Flight reservation

The `FlightReservation` class manages reservations made by customers against a specific flight instance. It has attributes like a unique reservation number, passengers and their assigned seats, and the reservation status.
The UML diagram for this class is given below:
R2: Customers should be able to reserve tickets for available flights and book multiple flights at once.

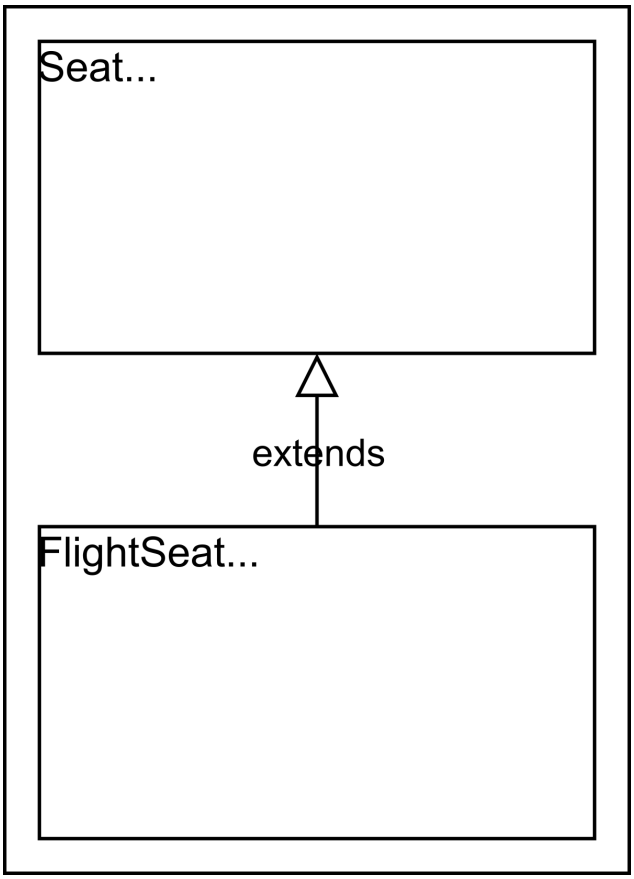


Figure 1.6: Seat and its derived class

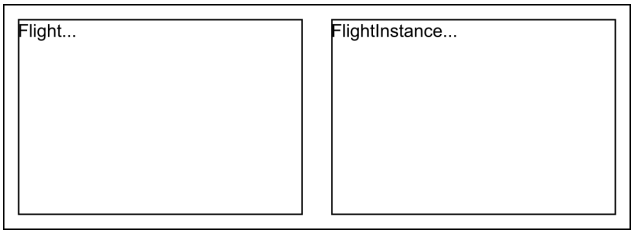


Figure 1.7: The Flight and FlightInstance classes

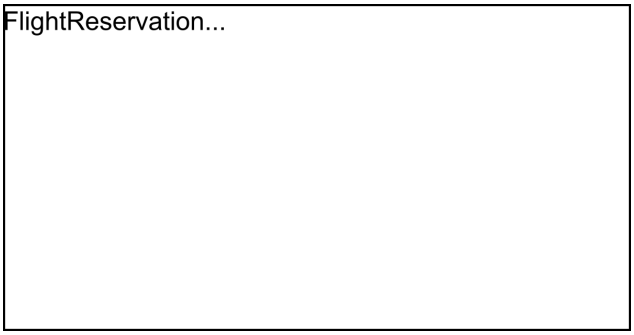


Figure 1.8: The FlightReservation class

Itinerary and passenger

Customers create an itinerary for their travel that contains flight reservations. The `Itinerary` class keeps track of all the reservations made, the list of passengers, and the departure/arrival airport.
The `Passenger` class maintains a record of all individuals with reservations. It also includes the basic travel information of the passenger.
Here's the class diagram for these classes:

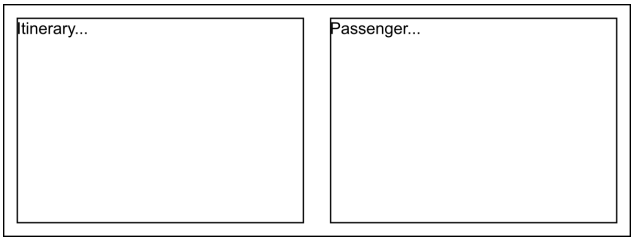


Figure 1.9: The Itinerary and Passenger classes

R2: Customers should be able to reserve tickets for available flights and book multiple flights at once.

Search and catalog

The `Search` interface allows customers to search for any flight using the following criteria:

- Departure and arrival date and time
- Source and destination airports

The `SearchCatalog` implements the search functionality and contains a list of all flights of the airline. The two classes are shown below:

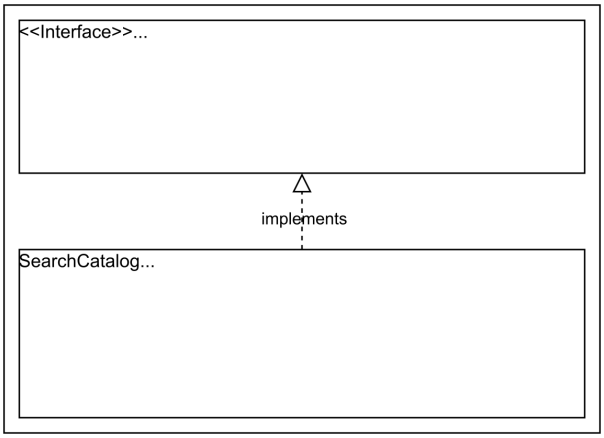


Figure 1.10: The Search interface and the SearchCatalog class

R1: Customers should be able to search for flights by date, departure, and destination airport.

Payment

The `Payment` class will be an abstract class and will have two child classes: `CreditCard`, and `Cash`.
The visual representation of these classes is as follows:

R9: The customer should be able to make payments against their flight reservations.

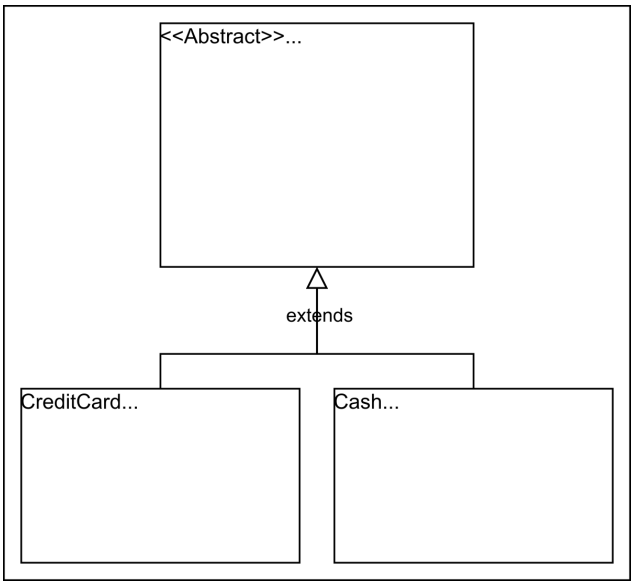


Figure 1.11: The Payment and its derived classes

Notification

`Notification` is an abstract class, since it can send a notification via email or SMS. It is primarily responsible for sending notifications as needed.
The UML representation of the class is shown below:

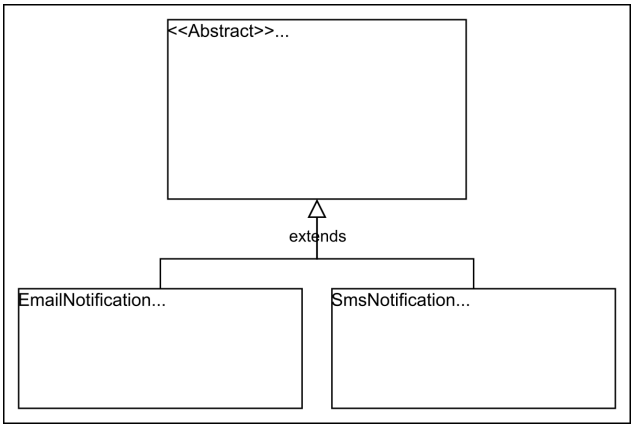


Figure 1.12: Notification and its derived classes

R13: The system should notify the customer whenever a reservation is made or canceled, or when there is an update regarding their flight.

Enumerations

The enumerations required in the airline management system design are listed below:

- `AccountStatus`**:** The account status indicates the current status of a customer's account—whether it is active, deactivated, closed, or blocked.
- `SeatStatus`**:** The seat status tells us whether the seat is available, booked, or a chance seat.

- `SeatType`: The seat type indicates whether the seat is of a regular, accessible, emergency exit, or extra-legroom type.
- `SeatClass`: The seat class indicates whether the seat is in Economy, Economy Plus, Business, or First Class.
- `FlightStatus`: The flight status tracks the flight instance, indicating whether it is active, scheduled, delayed, departed, landed, canceled, diverted, or unknown.
- `ReservationStatus`: The reservation status indicates the current status of the reservation—whether it is requested, pending, confirmed, checked in, or canceled.

PaymentStatus: The payment status indicates the current status of the payment—whether it is pending, completed, canceled, failed, declined, or refunded.

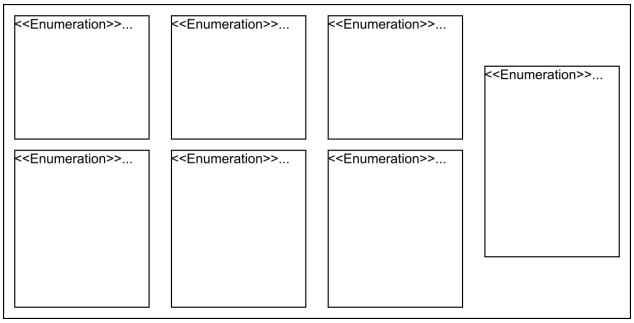


Figure 1.13: Enums in the airline management system

Custom data type

We need to create a custom data type, `Address`, that will store the physical location of any place.

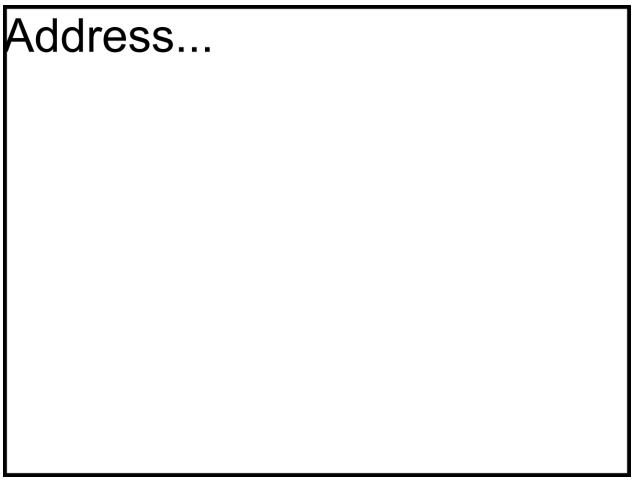


Figure 1.14: The Address custom data type

4.2 Relationship between classes

Now, we'll discuss the relationships between the classes we have defined in our airline management system above.

Association

The class diagram has the following association relationships:

One-way association

- Both `FrontDeskOfficer` and `Customer` classes have a one-way association with the `Itinerary` class.
- The `FlightReservation` class has a one-way association with the `FlightInstance`, `Payment`, and `Notification` classes.
- The `FlightSeat` class has a one-way association with the `FlightReservation` class.
- The `Person` class has a one-way association with the `Search` interface.

Two-way association

- The `Airport` class has a two-way association with the `Flight` class.
- Both the `Aircraft` and `Crew` classes have a two-way association with the `FlightInstance` class.
- The `Airline` class has a two-way association with the `Crew` class.

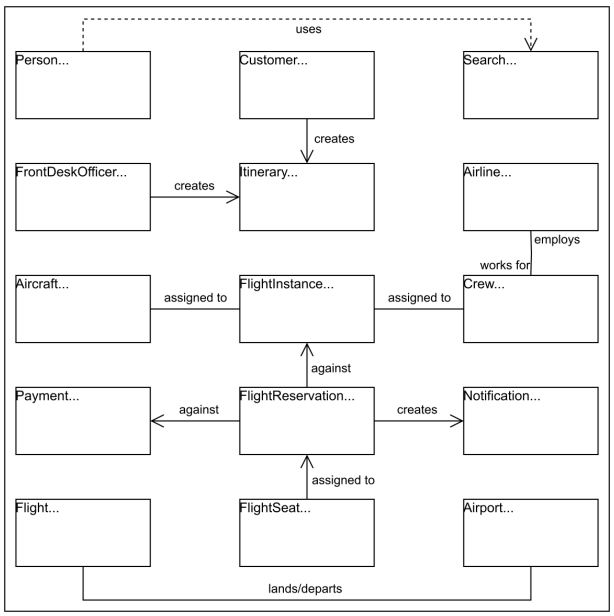


Figure 1.15: Association relationship between classes

Aggregation

The class diagram has the following aggregation relationship:

- The `Itinerary` class contains the `Passenger` class.

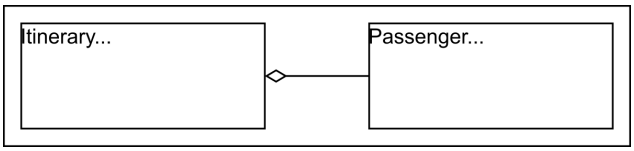


Figure 1.16: The aggregation relationship between classes

Composition

The class diagram has the following composition relationships:

- The `Person` class is composed of the `Account` class.
- The `Airline` class is composed of the `Aircraft` and `Flight` classes.
- The `Aircraft` class is composed of the `Seat` class.
- The `Flight` class is composed of the `FlightInstance` class.
- The `FlightInstance` class is composed of the `FlightSeat` class.
- The `Itinerary` class is composed of the `FlightReservaion` class.

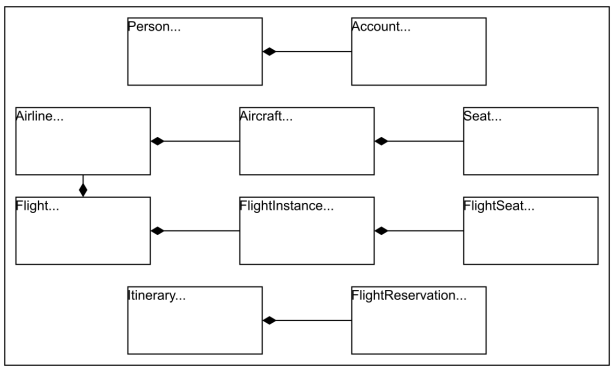


Figure 1.17: The composition relationship between classes

Inheritance

The following classes show an inheritance relationship:

- The `Admin`, `Crew`, `FrontDeskOfficer`, and `Customer` classes extend the `Person` class.
- The `FlightSeat` class extends the `Seat` class.
- The `CreditCard` and `Cash` classes extend the `Payment` class.
- The `EmailNotification` and `SmsNotification` classes extend the `Notification` class.

  Note: We have already discussed the inheritance relationship between classes in the component section above one by one.

4.3 Class diagram of the airline management system

In this section, we outline the multiplicity (cardinality) relationships between the main classes in our airline management system. For each relationship, we explain the allowed number of instances on each side and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Source	**Target**	**Multiplicity**	**Description**
Person	Account	1 – 1	Each person has one account
Customer	Person	1 – 1	Customer inherits from Person
Crew	Person	1 – 1	Crew inherits from Person
FrontDeskOfficer	Person	1 – 1	FrontDeskOfficer inherits from Person
Passenger	Person	1 – 1	Passenger inherits from Person
Flight	Airport (departure)	1 – 1	Each flight departs from one airport
Flight	Airport (arrival)	1 – 1	Each flight arrives at one airport
Flight	FlightInstance	1 – 0..*	A flight has multiple instances
FlightInstance	FlightSeat	1 – 0..*	A flight instance has multiple flight seats
FlightInstance	Flight	0..* – 1	Multiple flight instances relate to one flight
FlightInstance	Aircraft	1 – 1	Each flight instance uses one aircraft
FlightInstance	Crew	0..* – 0..*	Each flight instance can have multiple crew members
Aircraft	FlightSeat	1 – 0..*	Each aircraft has multiple flight seats
Aircraft	FlightInstance	1 – 0..*	An aircraft is used in multiple flight instances
FlightSeat	Seat	1 – 1	Flight seat is based on one seat
Seat	Aircraft	0..* – 1	A seat belongs to one aircraft
Airline	Flight	1 – 0..*	An airline operates multiple flights
Airline	Aircraft	1 – 0..*	An airline owns multiple aircrafts
Airline	Crew	1 – 0..*	An airline has multiple crew members
Itinerary	Airport (starting)	1 – 1	Itinerary starts at one airport
Itinerary	Airport (final)	1 – 1	Itinerary ends at one airport
Itinerary	FlightReservation	1 – 0..*	An itinerary includes multiple flight reservations
Itinerary	Passenger	1 – 0..*	An itinerary includes multiple passengers
FlightReservation	Passenger	1 – 0..*	A flight reservation includes multiple passengers
FlightReservation	FlightInstance	1 – 1	Each reservation is for one flight instance
FlightReservation	FlightSeat	1 – 1	Each reservation includes one flight seat
FlightReservation	Payment	1 – 1	Each reservation is paid with one payment
Payment	CreditCard	1 – 0..*	Payments can be made via credit card

Continued on next page

Table continued from previous page

Source	Target	Multiplicity	Description
Payment	Cash	1 – 0..*	Payments can be made via cash
Notification	EmailNotification	1 – 0..*	Notification via email
Notification	SmsNotification	1 – 0..*	Notification via SMS

Here's the complete class diagram for our airline management system:

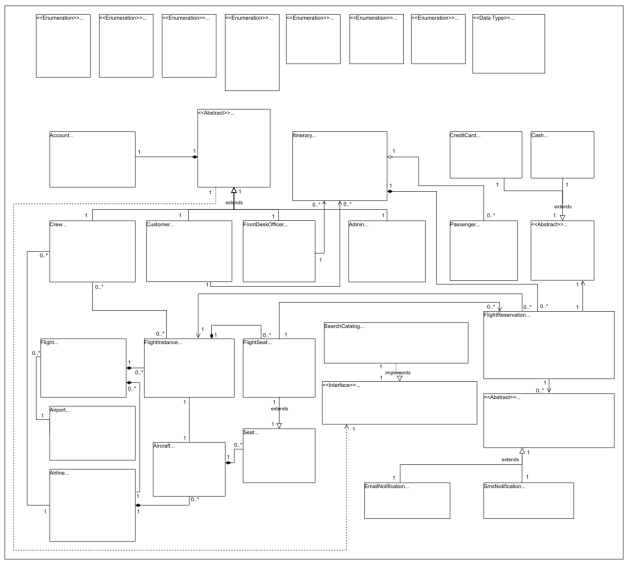


Figure 1.18: The class diagram of the airline management system

4.4 Design pattern

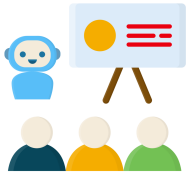
To implement the airline management system's core features in a flexible and scalable manner, we apply object-oriented design patterns based on the system's behavior.

- We know that the system sends notifications to users via email or SMS when a reservation is confirmed or a flight is delayed. To model this behavior, we can use the Observer design pattern.
- We know that the system allows customers to pay using various methods, such as credit cards or cash. To handle these different payment processes, we can use the Strategy design pattern.
- We know that notifications follow a common structure but differ in the way they are delivered (email vs. SMS). To enforce this structure while allowing specific implementations, we can use the Template Method design pattern.
- We know that only one instance of the search catalog is needed to manage the flight search system-wide. To ensure this, we can use the Singleton design pattern.
- We know that an itinerary may contain multiple flight reservations, and a reservation may include multiple passengers. To treat these collections uniformly, we can use the Composite design pattern.
- We know that creating an itinerary involves setting flights, passengers, and payments step by step. To simplify and manage this construction process, we can use the Builder design pattern.
- We know that actions like making a reservation, assigning a seat, or processing a payment need to be handled as discrete, trackable tasks. To encapsulate these actions, we can use the Command design pattern.

- We are aware that the system generates various types of notifications and payment objects. To manage this object creation logic cleanly, we can use the Factory design pattern.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



We have successfully designed the class diagram for the airline management system. Let 's see how we can construct the sequence diagram for the system in our next lesson.

5 Sequence Diagram for the Airline Management System

Sequence diagrams are an effective way to visualize the interactions between different entities and objects within a system. We can create different sequence diagrams for our airline management system. In this lesson, we will create sequence diagrams for the following two interactions:

- **Reserve a flight:** The customer reserves a flight online.
- **Sequence challenge:** Assign a seat via `FrontDeskOfficer`.

5.1 Reserve a flight

The sequence diagram for reserving a flight should have the following actors and objects that will interact with each other:

- **Actor:** `Customer`
- **Objects:** `SearchCatalog`, `FlightInstance`, `FlightSeat`, `FlightReservation`, `Itinerary`, `Payment`, `SmsNotification`, and `EmailNotification`.

Here are the steps in the reserve flight interaction:

1. The customer searches for flights flying from an airport on a particular date.
2. The catalog returns a list of flights that satisfy the search query.
3. The customer selects a flight.
4. Customer creates a reservation.
5. Customer creates an itinerary.
6. System requests payment.
7. Customer makes payment.

8. Customer notified.

   Note: We assume that the customer performs a valid operation and successfully reserves the seat.

Based on the order above, the sequence diagram for reserving a flight in the airline management system is given below.

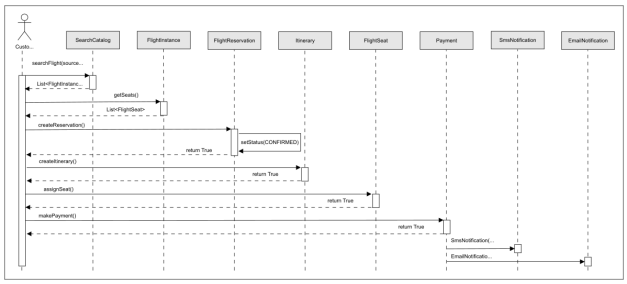


Figure 1.19: The sequence diagram for the "Reserve a flight" interaction

5.2 Sequence challenge: Assign seat via FrontDeskOfficer

Let's complete a sequence diagram for assigning a seat via `FrontDeskOfficer`. A skeleton of the sequence diagram is given below:

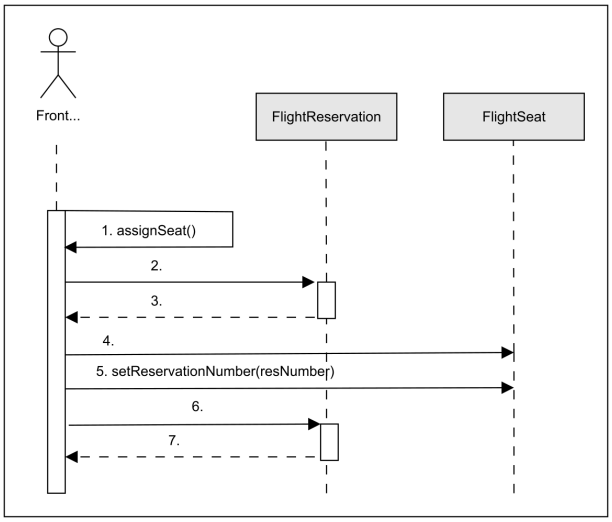


Figure 1.20: The sequence diagram for assigning a seat via FrontDeskOfficer

Notice that the arrows in the diagram above are numbered from 1 to 7. The message boxes shown below are the messages to be exchanged between the actor(s) and object(s). Can you rearrange the messages below in the correct sequence they should appear in, in the skeleton of the sequence diagram given above?

Note: If you're stuck, click the "Show Solution" button.

Next, let's examine the activity diagrams for the airline management system to understand the system's control flow.

6 Activity Diagram for the Airline Management System

Activity diagrams are an effective way to visualize the flow of messages from one activity to another within the system. There can be different activity diagrams that we can create for the airline management system. In this lesson, we will create activity diagrams for the following two activities:

- Creating an itinerary.
- **Activity challenge:** The user receives a confirmation notification after payment.

6.1 Create an itinerary

The states and actions involved in this activity diagram are provided below.

States

Initial state: The customer chooses an option to create the itinerary.
Final state: A customer books their ticket.

Actions

The customer selects an option to create an itinerary online or through the front desk officer. The customer then selects their flights. Finally, the customer books their ticket.

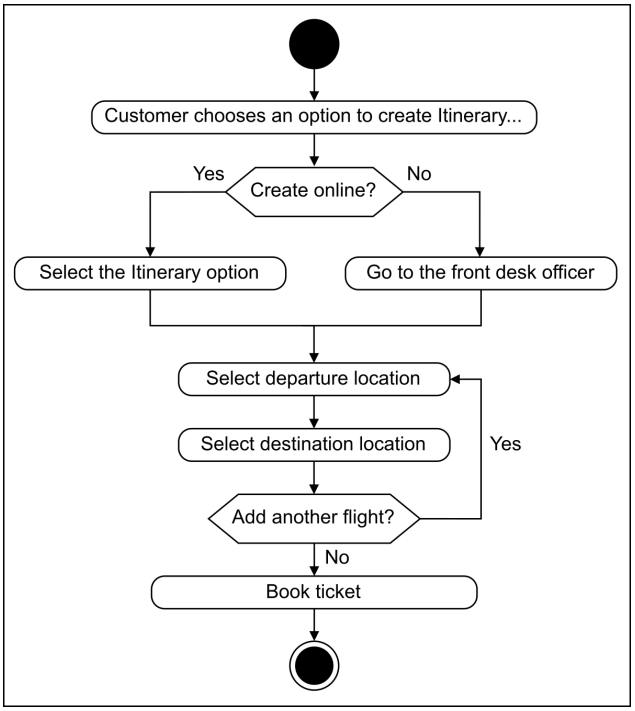


Figure 1.21: The activity diagram to create an itinerary

6.2 Activity challenge: The user receives a confirmation notification after payment

Let's create an activity diagram of a user receiving a confirmation notification after making the payment. A skeleton of the activity diagram is given below:

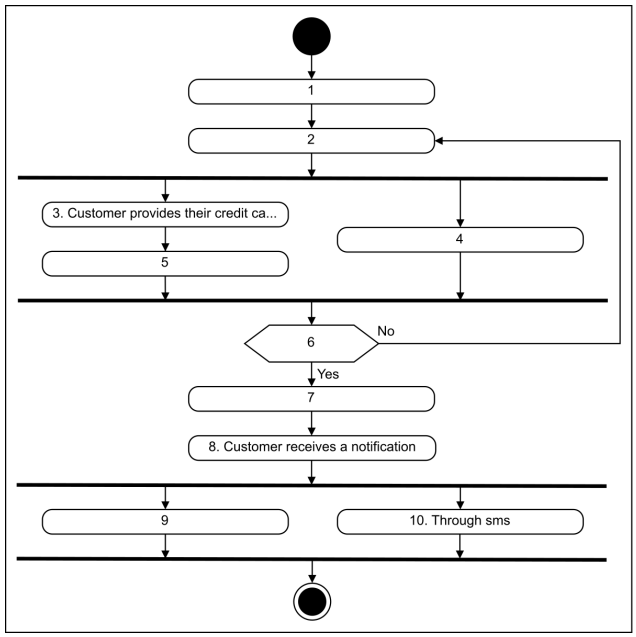


Figure 1.22: The activity diagram for a user receiving a confirmation notification after the payment

Notice that the actions in the diagram above are numbered from 1 to 10. The slots below represent the activities, and the arrows indicate the flow from one activity to another. Can you rearrange the slots below in the correct order they should appear in the activity diagram given above?

Note: If you're stuck, click the "Show Solution" button.

Alternatively, click the "Show complete diagram" button below to see the complete activity diagram.

We've looked at some of the activity diagrams of the airline management system. In the next lesson, we'll present the code for our designed classes in some of the most popular languages.

7 Code for the Airline Management System

We've reviewed various aspects of the airline management system and observed the attributes associated with the problem using different UML diagrams. Let 's explore the more practical side of things, where we will work on implementing the airline management system using multiple languages. This is usually the last step in an object-oriented design interview process.

We have chosen the following languages to write the skeleton code of the different classes present in the airline management system:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: Airline Management System.

7.1 The airline management system classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public methods.

Constants

The following code defines the various enums and custom data types being used in the airline management system:

> **Note:** JavaScript does not support enumerations, so we will use the Object.freeze() method as an alternative that freezes an object and prevents further modifications.

Constant definitions

```
public class Address {
  private int zipCode;
  private String streetAddress;
  private String city;
  private String state;
  private String country;
}

enum AccountStatus {
  ACTIVE,
  DISABLED,
  CLOSED,
  BLOCKED
}

enum SeatStatus {
  AVAILABLE,
  BOOKED,
  CHANCE
}

enum SeatType {
  REGULAR,
  ACCESSIBLE,
  EMERGENCY_EXIT,
  EXTRA_LEG_ROOM
}

enum SeatClass {
  ECONOMY,
  ECONOMY_PLUS,
  BUSINESS,
  FIRST_CLASS
}

enum FlightStatus {
  ACTIVE,
  SCHEDULED,
  DELAYED,
  LANDED,
  DEPARTED,
  CANCELED,
  DIVERTED,
  UNKNOWN
}

enum ReservationStatus {
  REQUESTED,
  PENDING,
  CONFIRMED,
```

```
  CHECKED_IN,
  CANCELED
}

enum PaymentStatus {
  PENDING,
  COMPLETED,
  FAILED,
  DECLINED,
  CANCELED,
  REFUNDED
}
```

Account and passenger

The `Account` class refers to an account for any user, including administrators, crew members, front desk officers, and customers. The `Passenger` class represents the passengers in the airline system. The definition of this class is given below:

The Account and Passenger classes

```
public class Account {
  private AccountStatus status;
  private int accountId;
  private String username;
  private String password;

  public boolean resetPassword();
}

public class Passenger {
  private int passengerId;
  private String name;
  private String gender;
  private Date dateOfBirth;
  private String passportNumber;
}
```

Person

`Person` is an abstract class that represents the various people or actors that can interact with the system. There are four types of persons: `Admin`, `Crew`, `FrontDeskOfficer`, and `Customer`. The implementation of the mentioned classes is shown below:

Person and its derived classes

```
public abstract class Person {
  private String name;
  private Address address;
  private String email;
  private String phone;
  private Account account;
}

public class Admin extends Person {
  public boolean addAircraft(Aircraft aircraft);
  public boolean addFlight(Flight flight);
  public boolean cancelFlight(Flight flight);
  public boolean assignCrew(Flight flight);
  public boolean blockUser(User user);
  public boolean unblockUser(User user);
```

```
}

public class Crew extends Person {
  public List<FlightInstance> viewSchedule();
}

public class FrontDeskOfficer extends Person {
  public List<Itinerary> viewItinerary();
  public boolean createItinerary();
  public boolean createReservation();
  public boolean assignSeat();
  public boolean makePayment();
}

public class Customer extends Person {
  private int customerId;

  public List<Itinerary> viewItinerary();
  public boolean createItinerary();
  public boolean createReservation();
  public boolean assignSeat();
  public boolean makePayment();
}
```

Seat and flight seat

The `Seat` and `FlightSeat` are used to keep track of the customer's seat. Here , `Seat` is the physical seat in the aircraft, and `FlightSeat` is the seat assigned to a specific flight instance. The definition of these two classes is given below:

The Seat and FlightSeat classes

```
public class Seat {
  private String seatNumber;
  private SeatType type;
  private SeatClass _class;
}

public class FlightSeat extends Seat {
  private double fare;
  private SeatStatus status;
  private String reservationNumber;
}
```

Flight and flight instance

The `Flight` and `FlightInstance` classes provide the details of the flight and its instances to the customer. The definition of these classes is given below:

The Flight and FlightInstance classes

```
public class Flight {
  private String flightNo;
  private int durationMin;
  private Airport departure;
  private Airport arrival;
  private List<FlightInstance> instances;
}

public class FlightInstance {
  private Flight flight;
```

```
  private Date departureTime;
  private String gate;
  private FlightStatus status;
  private Aircraft aircraft;
  private List<FlightSeat> seats;
}
```

Itinerary and flight reservation

The `Itinerary` and `FlightReservation` classes are used to track itineraries and flights reserved by customers. Both classes are defined below:

The Itinerary and FlightReservation classes

```
public class Itinerary {
  private Airport startingAirport;
  private Airport finalAirport;
  private Date creationDate;
  private List<FlightReservation> reservations;
  private List<Passenger> passengers;

  public boolean makeReservation();
  public boolean makePayment();
}

public class FlightReservation {
  private String reservationNumber;
  private FlightInstance flight;
  private HashMap<Passenger, FlightSeat> seatMap;
  private ReservationStatus status;
  private Date creationDate;

  public static FlightReservation fetchReservationDetails(String reservationNumber);
  public List<Passenger> getPassengers();
}
```

Payment

The `Payment` class is another abstract class with two child classes: `Cash` and `CreditCard`. This utilizes the `PaymentStatus` enum to track the payment status. The definition of this class is provided below:

Payment and its derived classes

```
public abstract class Payment {
  private int paymentId;
  private double amount;
  private PaymentStatus status;
  private Date timestamp;

  public abstract boolean makePayment();
}

public class Cash extends Payment {
    public boolean makePayment() {
        // functionality
    }
}

public class CreditCard extends Payment {
    private String nameOnCard;
    private String cardNumber;
```

```
    public boolean makePayment() {
        // functionality
    }
}
```

Notification

The `Notification` class is another abstract class responsible for sending notifications with two child classes: `SMSNotification` and `EmailNotification` . The implementation of this class is shown below:

Notification and its derived classes

```
public abstract class Notification {
    private int notificationId;
    private Date createdOn;
    private String content;

    public abstract void sendNotification(Account account);
}

class SmsNotification extends Notification {
    public void sendNotification(Account account) {
        // functionality
    }
}

class EmailNotification extends Notification {
    public void sendNotification(Account account) {
        // functionality
    }
}
```

Search and catalog

The `SearchCatalog` class contains flight instance information and implements the `Search` interface to enable search functionality based on the specified criteria. Both classes are defined below:

The Search interface and the SearchCatalog class

```
public interface Search {
  // Interface method (does not have a body)
  public List<FlightInstance> searchFlight(Airport source, Airport dest, Date arrival, Date
      departure);
}

public class SearchCatalog implements Search {
  private HashMap<Quartet<Airport, Airport, Date, Date>, List<FlightInstance>> flights;

  public List<FlightInstance> searchFlight(Airport source, Airport dest, Date arrival, Date
      departure) {
    // functionality
  }
}
```

Airport, aircraft, and airline

This section contains classes such as `Airport`, `Aircraft`, and `Airline` that comprise the infrastructure of our airline management system. Here , the `Airline` class is a Singleton. The definition of these classes is given below:

The Airport, Aircraft, and Airline classes

```java
public class Airport {
  private String name;
  private String code;
  private Address address;
  private List<Flight> flights;
}

public class Aircraft {
  private String name;
  private String code;
  private String model;
  private int seatCapacity;
  private List<Seat> seats;
}

public class Airline {
  private String name;
  private String code;
  private List<Flight> flights;
  private List<Aircraft> aircrafts;
  private List<Crew> crew;

  // The Airline is a singleton class that ensures it will have only one active instance at a time
  private static Airline airline = null;

  // Created a static method to access the singleton instance of Airline class
  public static Airline getInstance() {
    if (airline == null) {
      airline = new Airline();
    }
    return airline;
  }
}
```

7.2 Executable code: Airline management system

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the Airline Management System. The main driver code of the system resides in the `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the codes.

What does this code show

- **System initialization:** Initializes an admin, a customer, and a crew member, each with their own `Account`, `Address`, and role-specific objects. Two airports are set up, along with a Boeing 737 aircraft preloaded with seats. An initial flight is created between the two airports, and a flight instance is generated using the aircraft. This establishes the baseline for simulating operations such as booking, payments, and crew assignments.

- **Scenario 1 (Customer booking and payment):** The customer books a seat on the initialized flight by creating a reservation and associating it with a passenger. A payment is made using a credit card, and an email notification is sent upon successful payment, demonstrating the reservation and transaction flow through user interaction.

- **Scenario 2 (Admin flight and crew management):** The admin creates a new aircraft and adds a new flight using it. A flight instance is created, and the admin assigns the existing crew member to this instance. This highlights how administrators manage operational logistics, such as flights and personnel assignments.
- **Scenario 3 (Crew schedule viewing):** The crew member views their schedule through the `viewSchedule()` method, which retrieves their assigned flights via the admin's internal crew scheduling system. This simulates how crew members access up-to-date information about their assignments.

Hint: Try customizing the code!
This code is designed not just for reading, but for experimentation. You can edit, extend, or modify any part of the example.

7.3 Code executable

Executable Code: Airline Management System

```
import java.util.*;

public class Driver {

    public static void main(String[] args) {
        System.out.println("== Initializing system... ==");

        // Create addresses
        Address address1 = new Address(12345, "Street 1", "CityA", "StateA", "CountryA");
        Address address2 = new Address(67890, "Street 2", "CityB", "StateB", "CountryB");
        System.out.println("Addresses created.");

        // Airports
        Airport airport1 = new Airport("Alpha Airport", "ALP", address1, new ArrayList<>());
        Airport airport2 = new Airport("Beta Airport", "BET", address2, new ArrayList<>());
        System.out.println("Airports created: " + airport1.getName() + " and " + airport2.getName())
            ;

        // Aircraft and seats
        List<Seat> aircraftSeats = new ArrayList<>();
        for (int i = 1; i <= 10; i++) {
            aircraftSeats.add(new Seat("A" + i, SeatType.REGULAR, SeatClass.ECONOMY));
        }
        Aircraft aircraft = new Aircraft("Boeing 737", "B737", "737-800", 10, aircraftSeats);
        Airline.getInstance().addAircraft(aircraft);
        System.out.println("Aircraft created and added: " + aircraft.getName());

        // Accounts
        Account customerAccount = new Account(101, "jane_doe", "password");
        Account adminAccount = new Account(201, "admin1", "adminpass");
        Account crewAccount = new Account(301, "pilot_guy", "flyhigh");
        System.out.println("User accounts created.");

        // People
        Customer customer = new Customer(1, "Jane Doe", address1, "jane@example.com", "111-222-3333"
            , customerAccount);
        Admin admin = new Admin("Admin One", address2, "admin@example.com", "444-555-6666",
            adminAccount);
        Crew crew = new Crew("Captain Sky", address1, "pilot@example.com", "777-888-9999",
            crewAccount);
        Airline.getInstance().addCrew(crew);
        System.out.println("Customer, Admin, and Crew created.");

        // Flight and flight instance
        Flight flight = new Flight("FL123", 120, airport1, airport2, new ArrayList<>());
        FlightInstance flightInstance = new FlightInstance(flight, new Date(), "G5", FlightStatus.
            SCHEDULED, aircraft, new ArrayList<>());

        for (Seat seat : aircraftSeats) {
```

```
            flightInstance.getSeats().add(new FlightSeat(seat.getSeatNumber(), seat.getType(), seat.
                getSeatClass(), 150.0));
        }
        flight.getInstances().add(flightInstance);
        admin.addFlight(flight);
        System.out.println("Initial flight created and flight instance added.");

        System.out.println("System initialized.
");


        // ========= Scenario 1 =========
        System.out.println("== Scenario 1: Customer books and pays ==");

        Passenger p1 = new Passenger(1, "Jane Doe", "Female", createDate(1990, 5, 5), "P12345");
        FlightSeat selectedSeat = flightInstance.getSeats().get(0);
        selectedSeat.setStatus(SeatStatus.BOOKED);
        selectedSeat.setReservationNumber("RESV001");

        HashMap<Passenger, FlightSeat> seatMap = new HashMap<>();
        seatMap.put(p1, selectedSeat);

        FlightReservation reservation = new FlightReservation("RESV001", flightInstance, seatMap,
            ReservationStatus.CONFIRMED, new Date());

        List<FlightReservation> reservations = new ArrayList<>();
        reservations.add(reservation);
        List<Passenger> passengers = new ArrayList<>();
        passengers.add(p1);

        Itinerary itinerary = new Itinerary(airport1, airport2, new Date(), reservations, passengers
            );
        itinerary.makeReservation();

        CreditCard card = new CreditCard(1, selectedSeat.getFare(), "Jane Doe", "4111111111111111");
        customer.makePayment(card);

        System.out.println();

        // ========= Scenario 2 =========
        System.out.println("== Scenario 2: Admin adds a new flight and assigns crew ==");

        Aircraft newAircraft = new Aircraft("Airbus A320", "A320", "A320-200", 5, new ArrayList<>())
            ;
        admin.addAircraft(newAircraft);

        Flight newFlight = new Flight("FL456", 180, airport2, airport1, new ArrayList<>());
        FlightInstance newInstance = new FlightInstance(newFlight, new Date(), "H2", FlightStatus.
            SCHEDULED, newAircraft, new ArrayList<>());
        newFlight.getInstances().add(newInstance);

        admin.addFlight(newFlight);
        admin.assignCrew(crew, newInstance);

        System.out.println();

        // ========= Scenario 3 =========
        System.out.println("== Scenario 3: Crew views schedule ==");

        List<FlightInstance> crewSchedule = crew.viewSchedule();
        for (FlightInstance fi : crewSchedule) {
            System.out.println("Assigned to Flight " + fi.getFlight().getFlightNo() + " at Gate " +
                fi.getGate());
        }
    }

    // Utility method to safely create a Date from year, month, day
    private static Date createDate(int year, int month, int day) {
        Calendar cal = Calendar.getInstance();
        cal.set(year, month - 1, day); // Java months are 0-based
        return cal.getTime();
    }
```

```
}
```

7.4 Wrapping up

We've explored the complete design of the airline management system in this chapter. We 've looked at how a basic airline management system can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing an Airline Management System

After wrapping up the airline management system case study, it's helpful to review and consolidate your learning. We 'll revisit the key system requirements, break down the primary classes and their responsibilities, and discuss the important design principles that guided the solution. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how each component contributes to the system's functionality.

8.1 Key requirements

The following are the core functional and nonfunctional requirements that define the airline management system:

1. Customers can search for flights by date, departure, and destination airport.
2. Customers can reserve tickets for available flights, including booking multiple flights or seats at once.
3. Customers can view flight details, including available seats, schedules, and departure/arrival times.
4. Admin can add, update, or cancel scheduled flights, as well as add aircraft to the system.
5. Airlines can operate flights from different airports and manage multiple aircraft.
6. Admin can assign pilots and crew to flights.
7. Customers must be able to make payments and cancel their reservations.
8. Front desk officers can reserve tickets, create itineraries, and make payments on behalf of customers.
9. Flight crew members can view their assigned flight schedules.
10. The system sends notifications to customers about reservations, cancellations, and flight updates.

8.2 System actors

The main actors interacting with the airline management system are:

- **Customer:** Searches for flights, creates itineraries, reserves/cancels tickets, and manages bookings.
- **Front desk officer:** Performs all customer actions on their behalf, including reservations and payments.
- **Admin:** Manages flights, aircraft, crew assignments, and overall system operations.

8.3 Important classes and relationships

Below is a summary of the key classes, their purposes, and their major relationships within the system:

Class	Description	Important Relationships
Account	Stores usernames and ID for system users.	Composed within Person
Admin	Manages flights, aircraft, crew, and system records.	Inherits from Person
FrontDeskOfficer	Handles reservations, payments, and itineraries for customers.	Inherits from Person; Associates with Itinerary
Airline	Main organizational class: holds details of the airline.	Composed of Aircraft, Flight; Associated with Crew
Airport	Represents an airport with unique attributes.	Two-way association with Flight
Aircraft	Represents an airplane, with model and seat details.	Composed of Seat; Associated with FlightInstance
Seat	Represents a physical seat in the aircraft.	Parent class to FlightSeat; Composed in Aircraft
FlightReservation	Handles customer reservations against flight instances.	Associated with FlightInstance, Payment, Notification
Itinerary	Represents a customer's travel plan (can include multiple reservations).	Composed of FlightReservation; Contains Passenger
Passenger	Represents each individual traveling under an itinerary.	Aggregated in Itinerary
Search	Interface for searching flights based on various criteria.	Implemented by SearchCatalog; Associated with Person
Payment	Handles payment processes for reservations.	Parent to CreditCard, Cash; Associated with FlightReservation

8.4 Design highlights

This section highlights the major design choices and rationale behind the system structure.

8.5 Design approach

The system employs a bottom-up approach, starting with the smallest building blocks, such as seats and flight reservations, which are then composed to create larger, more complex components, including aircraft, flights, itineraries, and the entire management system.

8.6 Design principles

The design follows key SOLID principles:

- **Single Responsibility principle (SRP):** Each class is focused on one aspect, e.g., `FlightSeat` for seat allocation, `Payment` for transaction processing, `Notification` for alerts.
- **Open/Closed principle (OCP):** The system allows adding new payment types, seat classes, or notification mechanisms without altering existing code.
- **Liskov Substitution principle (LSP):** Subtypes like `Admin`, `Crew`, `FrontDeskOfficer`, and `Customer` can substitute their base `Person` class.
- **Interface Segregation principle (ISP):** Search, payment, and notification features are separated into distinct interfaces and implementations.
- **Dependency Inversion principle (DIP):** High-level modules depend on abstractions (e.g., `Payment`, `Notification`) rather than specific implementations.

8.7 Design patterns

The system leverages several established design patterns:

- **Singleton pattern:** Ensures only one `Airline` instance is available throughout the application.
- **Observer pattern:** Enables passengers to be automatically notified of any changes to their flight reservations or status.

8.8 Object interactions

The system's dynamic behavior is defined by how its core objects collaborate.
Flight reservation:

- The customer searches for flights via `SearchCatalog` using date and airport filters.
- The system returns available flights.
- The customer selects a flight and creates a reservation, which creates a `FlightReservation` and assigns a `FlightSeat`.
- The customer (or officer) creates an `Itinerary` that bundles one or more reservations.
- The system prompts payment; a `Payment` object is created and processed.
- On success, the system triggers `SmsNotification` and `EmailNotification` to confirm the booking.

Seat assignment (Front desk):

- The front desk officer assigns passengers seats for a specific flight.
- The chosen `FlightSeat` is associated with the passenger's reservation and marked as booked.

8.9 System workflow

This section outlines the end-to-end scenarios and main life cycle events for users and the airline:
Customer journey:

- Customer or officer logs in and chooses to create an itinerary.
- Searches for flights, selects preferred options, and adds them to the itinerary.
- Proceeds to payment; once payment is made, receives confirmation notifications.
- Can later update or cancel reservations, with the system handling refunds and notifications.

Admin actions:

- Adds new aircraft, schedules new flights, and updates flight instances.
- Assigns pilots and crew to flights and can block or unblock users.

9 Key Insights and Tips: Designing an Airline Management System

Great job on wrapping up the airline management system case study! This lesson'll build on your progress by spotlighting frequent design pitfalls and sharing targeted interview tips. To help you test your understanding, you'll also find a quick quiz, plus a selection of related case studies to further strengthen your object-oriented design skills.

9.1 Common mistakes

Avoiding these mistakes will help you create a more robust and scalable airline management system:

- **Mixing concerns:** Don't combine booking, payments, flight details, and notifications in a single class. Modularize your design—separate entities like `Flight`, `Seat`, `Itinerary`, `Payment`, and `Notification`.
- **Inconsistent seat assignment:** Failing to validate seat assignments can lead to double-booking. Always ensure one seat maps to one passenger for each flight instance.
- **Neglecting payment and refund flows:** Failing to account for various payment methods, payment statuses, and refund logic can result in a suboptimal user experience. Model payments as abstract classes with appropriate status tracking.
- **Overcomplicating relationships between classes:** Creating overly complex, tightly coupled class relationships (for example, making Payment directly manage seat allocation) can make the system hard to maintain and extend. Always favor clear, well-defined associations and use aggregation/composition appropriately.
- **Ignoring real-time notifications:** Failing to implement notifications for booking, cancellations, or status updates can leave users confused or uninformed. Utilize a notification system (e.g., Observer pattern) to receive timely alerts.

9.2 Interview tips

Understand key design patterns, system components, and real-world applications to prepare for your interview and confidently discuss your approach to these concepts.

Tip	Explanation
Demonstrate how your design accommodates change.	Illustrate how your architecture accommodates future needs, such as internationalization, integration with external APIs, or regulatory changes, showing flexibility without sacrificing integrity.
Discuss error handling and transaction safety.	Explain your approach to ensuring data consistency in critical business flows, such as seat reservation and payment, including mechanisms like transaction boundaries and rollback strategies.
Demonstrate the use of appropriate design patterns.	Discuss why you chose certain patterns (e.g., Observer for notifications, Factory for payment methods).
Discuss extensibility.	Explain how your design supports adding new features, roles, or payment methods in the future.
Explain pricing and seat logic.	Be prepared to answer questions about dynamic pricing, seat classes, and reservation constraints.

9.3 Test your understanding

Now that you understand OOD principles well, solve the quiz below to test your knowledge.

Airline Management System

Question 1: Which design pattern would you use to add new payment types (e.g., Apple Pay, PayPal) to the system without modifying the payment processing logic?

- Singleton pattern
- **[CORRECT]** Factory pattern
  Explanation: Factory pattern allows you to introduce new payment classes without changing the core logic, promoting extensibility and OCP (Open/Closed Principle).
- Observer pattern
- Adapter pattern

Question 2: What OOD concept helps you share core flight search and reservation logic among different user types (Customer, Front Desk Officer) while allowing for custom behaviors?

- Multiple inheritance
- Interface segregation
- **[CORRECT]** Use of interfaces and composition
  Explanation: Interfaces (or abstract classes) allow shared logic, while composition lets you add custom behaviors where needed, promoting reusability and maintainability.
- Hardcoding logic in each class

Question 3: Which design pattern is best for sending users real-time booking and cancellation notifications?

- Factory pattern
- **[CORRECT]** Observer pattern
  Explanation: The Observer pattern is the best choice because it enables the system to automatically notify all registered observers (like email or SMS services) whenever a booking or cancellation event occurs.
- Builder pattern
- Prototype pattern

Question 4: What design approach would you use to support multiple seat types (such as regular, emergency exit, and extra legroom) and ensure that future seat categories can be added easily?

- **[CORRECT]** Use an enumeration and class hierarchy for seat types.
  Explanation: Combining enums for categorization and a class hierarchy for behavior allows for both easy extension and clear logic for each seat type.
- Hardcode seat types in the `Seat` class.
- Store seat type as a plain string.

- Require manual code changes for each new seat type.

Question 5: How does the system ensure that two customers cannot be assigned the same seat on a flight?

- Manual review of each reservation
- **[CORRECT]** Database constraints or transaction locking during seat assignment
  Explanation: This prevents race conditions and ensures no two reservations are made for the same seat at the same time, even with concurrent bookings.
- Sending an SMS alert to the customer
- Letting the crew handle seating at check-in

9.4 Related case studies

The following case studies will help reinforce concepts used in the Airline Management System:

- **Design of a library management system:** Emphasizes user roles, reservation management, and notification flows—translatable to airline contexts.
- **Design of a parking lot system:** Demonstrates entity management, slot assignments, and payment workflows, with clear analogies to seat management and bookings in airlines.
- **Design of a hotel reservation system:** Focuses on bookings, payments, room types, and notifications.
- **Design of a ride sharing platform:** Covers scheduling, reservations, dynamic pricing, and trip management.
- **Design of a ticketing system for events:** Includes seat assignments, reservations, payments, and cancellations.